

Received June 11, 2019, accepted June 21, 2019, date of publication June 26, 2019, date of current version July 16, 2019.

Digital Object Identifier 10.1109/ACCESS.2019.2924960

A Simple and Efficient Network for Small Target Detection

MORAN JU^{1,2,3,4,5}, JIANGNING LUO⁶, PANPAN ZHANG^{1,2,3,4,5},
MIAO HE^{1,2,3,4,5}, AND HAIBO LUO^{1,2,4,5}

¹Shenyang Institute of Automation, Chinese Academy of Sciences, Shenyang 110016, China

²Institutes for Robotics and Intelligent Manufacturing, Chinese Academy of Sciences, Shenyang, 110016, China

³University of Chinese Academy of Sciences, Beijing 100049, China

⁴Key Laboratory of Opt-Electronic Information Processing, Chinese Academy of Sciences, Shenyang 110016, China

⁵The Key Laboratory of Image Understanding and Computer Vision, Shenyang 110016, China

⁶Department of Electrical and Computer Engineering, McGill University, Montreal, QC H3A 0G4, Canada

Corresponding author: Moran Ju (jumoran@sia.cn)

ABSTRACT Target detection based on deep learning is developing rapidly. However, small target detection is still a challenge. In this paper, a simple and efficient network for small target detection is proposed. We put forward to improve the detection performance of the small targets in three aspects. First, as the contextual information is important to detect the small targets, we proposed to use “dilated module” to expand the receptive field without loss of resolution or coverage. Second, we applied feature fusion in different dilated modules to improve the ability of the network in detecting small targets. Finally, we used “passthrough module” to get the finer-grained information from the earlier layer and combined it with the semantic information from the deeper layer. To improve the detection speed of the network, it is proposed to use 1×1 convolution to reduce the dimension of the network. We composed small vehicle dataset based on VEDAI dataset and DOTA dataset, respectively, and also analyzed the distribution of the small targets in each dataset. To evaluate the performance of the proposed network, we trained the model on the dataset above and compared with the state-of-the-art target detection algorithms, our approach achieved 80.16% average precision (AP) on VEDAI dataset and 88.63% AP on DOTA dataset and the frames per second (FPS) is 75.4. The AP of our network is much better than the result of the tiny YOLO V3 and is nearly the same as the result of the YOLO V3. However, the FPS of our network is almost the same as that of the tiny YOLO V3.

INDEX TERMS Deep learning, target detection, passthrough layer, dilated convolution.

I. INTRODUCTION

Target detection has become a hot research topic in the field of computer vision. The goal of target detection is to determine the spatial location and the category of the targets. At present, target detection has been widely used in our daily life [1], [2], such as pedestrian detection [3], face detection [4], [5], vehicle detection [6]–[8], intelligent surveillance [9]–[11] and autonomous driving [12], [13], etc.

As one of the key technologies of target detection, small target detection has become a research hotspot in recent years. Small targets are characterized by low resolution and inconspicuous features which causes low detection rate and high false alarm rate. Therefore, small target detection is still one of the open challenges in target detection [14].

The associate editor coordinating the review of this manuscript and approving it for publication was Bora Onat.

With the development of deep learning, a variety of target detection approaches based on deep learning have been proposed. The deep convolution neural network can learn the features of the targets to be detected with the dataset autonomously and improve the performance of its model gradually. The widely used target detection approaches based on deep learning can be divided into two categories. The first category is the two stage target detection approach, such as R-CNN [15], Fast R-CNN [16], Faster R-CNN [17], Mask R-CNN [18], etc. The target detection process is divided into two phases by these algorithms. Firstly it generates a sparse of candidate targets and extract the feature information of the candidate targets. Then the location and category of the candidate targets can be further predicted and identified. These approaches are not suitable for some real-time situations. The second category is the one stage target detection algorithm, such as OverFeat [19], SSD [20], DSSD [21], YOLO [22], YOLO 9000 [23], YOLO V3 [24],

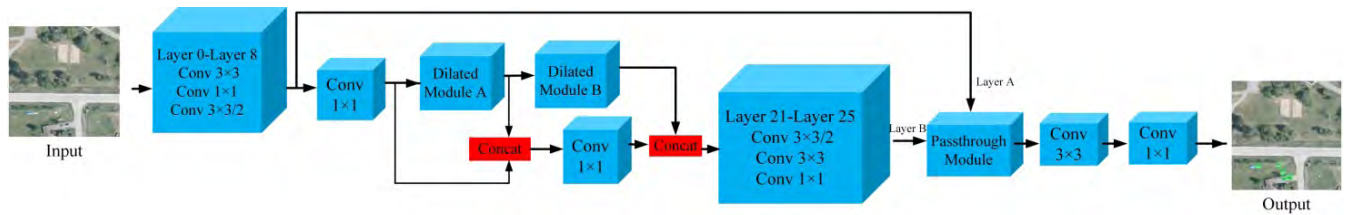


FIGURE 1. The framework of the proposed network. Concatenation is used between dilated module A and module B. Then, feature map downsampled twice from the input is used as finer-grained information and combined with the feature map downsampled three times by passthrough module.

You Only Look Twice [25], etc. There is no need to generate the candidate targets first. These approaches predict the target location and category directly through the network. Therefore, one stage target detection approach has faster detection speed.

Although target detection has been the subject of research for a long time, the problem of detecting small targets is largely ignored. Small object RCNN [26] introduces a small dataset and selects anchor box with small sizes to detect the small targets. They improve the detection performance of small targets by upsampling the images and cropping the corresponding context region enclosing the proposal region. [27] applies Faster RCNN to the task of company logo detection. They introduce an improved scheme for generating anchor proposals and propose a modification to Faster RCNN which leverages higher-resolution feature maps for small targets. The methods above are based on two stage target detection approach. They are not suitable for some real-time situations. [28] explores three aspects of the problem in the context of finding small faces: the role of scale invariance, image resolution, and contextual reasoning. However, it is not a real-time detection model and only used in detecting face.

As one of the state-of-the-art object detection approaches, YOLO V3 can detect targets of different scales with high accuracy. There are 3 scales in YOLO V3. So it can detect both big targets and small ones. Because YOLO V3 merges the features from earlier layer with the upsampled features from the deeper layer, its detection performance for small targets is excellent. To improve the detection speed, a network called tiny YOLO V3 is proposed. However, compared with YOLO V3, its performance to detect small targets is poor. The framework of our network is shown in Fig. 1. We explore three aspects to improve the detection ability of the small targets: dilated module, feature fusion, and passthrough module.

Dilated Module: The contextual information is important to detect the small targets [28]. One approach involves repeated upsampling to recover the lost information while expanding the receptive field with downsampled layers. However, some information of the targets is inevitably lost in this process. Inspired by dilated convolution [29], we proposed to use ‘dilated module’ to expand the receptive field without loss of resolution or coverage.

Feature Fusion: Feature map from different layer contains different feature information [30]. The features from the earlier layer contain finer-grained information and the features

from the deeper layer contain more meaningful semantic information. Both the features from the earlier and the deeper layers are important to detect the small targets in an image. Therefore, we applied feature fusion in different dilated modules by concatenation to make good use of the features of the small targets.

Passthrough Module: The location information is vital to small targets. We proposed to use ‘passthrough module’ to get the accurate location information of the small targets from the earlier feature map and combined it with the semantic information from the deeper feature map.

To reduce the amount of computation and improve the detection speed, it is proposed to use 1×1 convolution to reduce the dimension of the network. Finally, we trained and evaluated our model on the dataset we made based on VEDAI dataset [31] and DOTA dataset [32]. The comparative experiment shows that the mean average precision (mAP) of our network is much better than the result of tiny YOLO V3 and is nearly the same as the result of YOLO V3. However, the frames per second (FPS) of our network is almost the same as the FPS of tiny YOLO V3. All results above demonstrated that our network is really simple, fast and efficient.

The main contributions of our work can be summarized as follows:

(1) We put forward ‘dilated module’ to expand the receptive field and ‘passthrough module’ to make good use of the contextual information and the location information of the small targets. Considering the characteristics of the small targets, feature fusion is applied to get both of the rich details and semantic information from the image. To make the network efficient and fast, 1×1 convolution is used to reduce the dimension of the network.

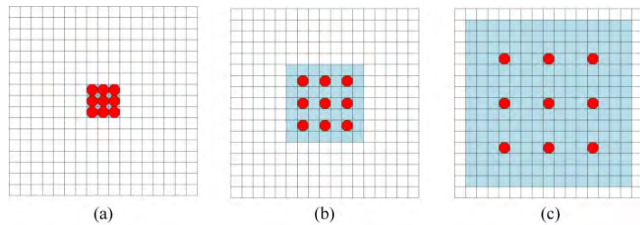
(2) We composed small vehicle dataset based on VEDAI dataset and DOTA dataset respectively and also analyzed the distribution of the small targets in each dataset.

(3) We trained and evaluated the model on VEDAI dataset and DOTA dataset. The results show that our proposed network is really simple, fast and efficient. Both quantitative and qualitative comparisons of our network with the state-of-the-art networks are provided.

The rest of this paper is organized as follows. Section 2 explores three aspects to improve the detection performance of the small targets and introduces the main framework of our network in detail. Section 3 makes analysis

TABLE 1. Comparison of receptive field between normal convolution and dilated convolution.

Layer	Network	Size/Stride	RF	Layer	Network	Size/Stride/Dilated rate	RF
0	Convolutional	3×3/1	3×3	0	Dilated Convolutional	3×3/1/1	3×3
1	Convolutional	3×3/1	5×5	1	Dilated Convolutional	3×3/1/2	7×7
2	Convolutional	3×3/1	7×7	2	Dilated Convolutional	3×3/1/4	15×15

**FIGURE 2.** Dilated convolution. (a) Dilated rate = 1. (b) Dilated rate = 2; (c) Dilated rate = 4.

of the dataset and demonstrates the experiments and the results while section 4 is the conclusion of this paper.

II. PROPOSED METHODS

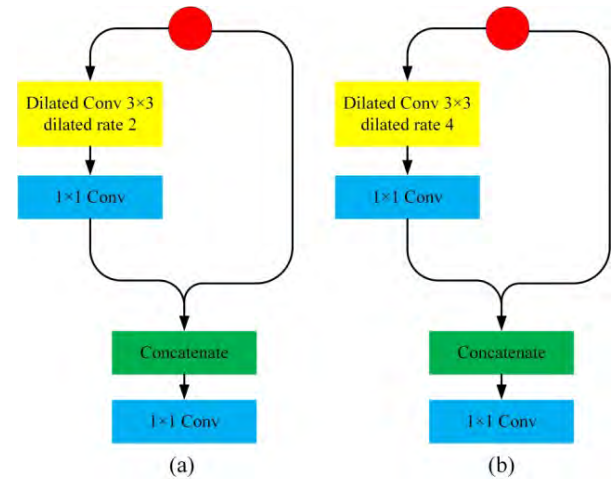
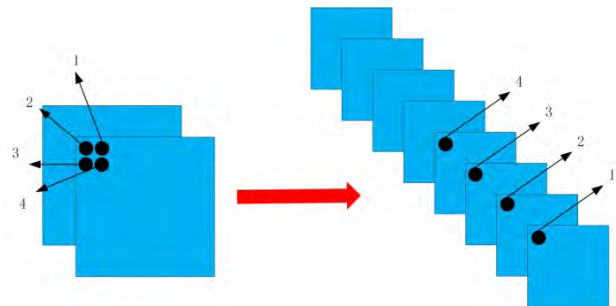
We used ‘dilated module’ to expand the receptive field. To locate the small targets accurately and aggregate multi-scale contextual information, ‘passthrough module’ is used to combine finer-grained information with semantic information. Feature fusion is used to improve the ability of network in detecting small targets. To reduce the amount of computation and improve the detection speed, it is proposed to use 1×1 convolution to reduce the dimension of the network.

A. DILATED MODULE

Dilated Module: Generally, contextual information is very useful for improving the small target detection performance. To get more contextual information, it is necessary to expand the receptive field of the network. Using downsampling can expand the receptive field. However, the size of the feature map will be reduced which causes the loss of the information about the small targets. Dilated convolution can expand the receptive field by increasing the kernel size of the convolution with different dilated rate. The size of the feature maps is not changed. So it supports exponential expansion of the receptive field without loss of resolution and coverage [27]. As shown in Fig. 2.

Fig. 2(a) is produced by a 1-dilated convolution and its receptive field is 3×3 . Fig. 2(b) is produced from Fig 2(a) by a 2-dilated convolution and its receptive field is 7×7 . Fig. 2(c) is produced from Fig. 2(b) by a 4-dilated convolution and its receptive field is 15×15 . The comparison of receptive field between normal convolution and dilated convolution is shown in Table 1. With dilated convolution, the receptive field of the network is obviously expanded.

We used dilated convolution as basic element to build ‘dilated module’. To reuse the features in the earlier layer,

**FIGURE 3.** Dilated module. (a) Module A with a 2-dilated convolution. (b) Module B with a 4-dilated convolution.**FIGURE 4.** The implementation of passthrough layer.

we merge the features from the earlier layer and the ones from the deeper layer by concatenation. To make the network efficient and fast, we used 1×1 convolution to reduce the dimension of the module. As shown in Fig. 3. Fig. 3(a) is dilated module A with 2-dilated convolution. Fig. 3(b) is dilated module B with 4-dilated convolution.

B. PASSTHROUGH MODULE

The features from the earlier layer contain finer-grained information which is useful to determine the location of small targets. To utilize the features from the earlier layer, we used passthrough layer whose stride is 2. The passthrough layer can transform the feature map from $2N \times 2N \times C$ to $N \times N \times 4C$. The process can be shown in Fig. 4. The left is the input of the passthrough layer and the right is the output of the passthrough layer. The number represents the pixel in the feature map.

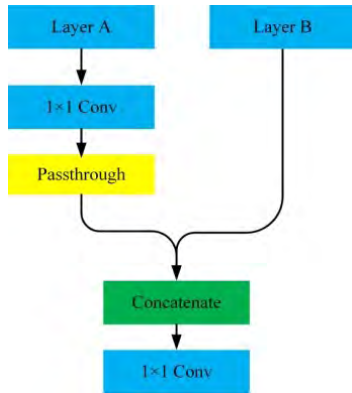


FIGURE 5. Passthrough module.

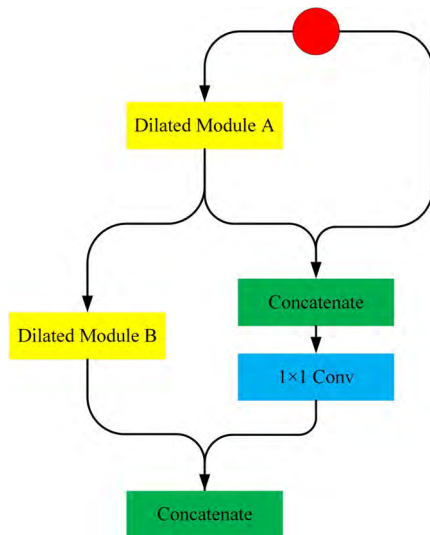


FIGURE 6. Feature fusion between different dilated modules.

We used passthrough layer as basic element to build ‘passthrough module’. With the ‘passthrough module’, the features from the earlier layer can merge with the deeper layer. The 1×1 convolution is used to reduce the dimension of the module. As shown in Fig. 5. Layer A represents the earlier layer and Layer B represents the deeper layer.

C. FEATURE FUSION

In this paper, concatenation is used to merge the features from the earlier layer with the features from the deeper layers. There are two kinds of feature fusion methods in the framework of the proposed network. One is concatenating the feature maps between different dilated modules as shown in Fig. 6. Because the dilated modules haven’t changed the size of the feature map, the features can be merged by concatenation directly. We fused features from dilated modules A and module B to enhance the feature information with different receptive fields. Feature fusion of this kind is useful for feature reuse.

The other is like passthrough layer. As the feature map undergoes the downsampled layers, the size of the feature

	Type	Filters	Size/Stride	Output
1×	Convolutional	16	3×3/1	512×512
	Convolutional	32	3×3/2	256×256
	Convolutional	32	1×1	
	Convolutional	64	3×3	
4×	Residual			256×256
	Convolutional	128	3×3/2	128×128
	Convolutional	64	1×1	
	Convolutional	128	3×3	
8×	Residual			128×128
	Convolutional	256	3×3/2	64×64
	Convolutional	128	1×1	
	Convolutional	256	3×3	
8×	Residual			64×64
	Convolutional	512	3×3/2	32×32
	Convolutional	256	1×1	
	Convolutional	512	3×3	
4×	Residual			32×32
	Convolutional	1024	3×3/2	16×16
	Convolutional	512	1×1	
	Convolutional	1204	3×3	
3×	Residual			16×16
	Convolutional	512	1×1	
	Convolutional	1204	3×3	
	Yolo Route			
3×	Convolutional	256	Layer 79 1×1	32×32
	Upsample			
	Route		Layer 85+Layer 61	
	Convolutional	512	1×1	
3×	Convolutional	1024	3×3	32×32
	Yolo Route			
	Convolutional	128	Layer 91 1×1	64×64
	Upsample			
3×	Route		Layer 97+Layer 36	
	Convolutional	512	1×1	
	Convolutional	1024	3×3	64×64
	Yolo			

FIGURE 7. The architecture of YOLO V3.

map has been changed. If we want to merge the features from different feature maps by concatenation, we have to unify the size of the feature map by passthrough layer or upsampled layer. Feature fusion of this kind can merge the finer-grained information from earlier layer with the meaningful semantic information from the downsampled features. In this paper, to merge the finer-grained information from earlier layer without loss of resolution, passthrough layer is selected.

D. NETWORK ARCHITECTURE

The architecture of YOLO V3 is shown in Fig. 7. The architecture of our network and Tiny YOLO V3 is shown in Table 2.

YOLO V3 draws on the idea of the residual neural network and there are 53 convolutional layers in the network to extract the features of the small targets. To detect the small targets, the deeper layers are concatenated with the earlier layers with upsampling. So it has good performance of detecting small targets. However, the YOLO V3 network is too deep and the model size is much bigger than our proposed network. Tiny YOLO V3 is simplified by YOLO V3. There are 5 max pooling layers which results in the loss of the information of the small targets and makes it difficult to locate the small targets accurately. So its performance of detecting small

TABLE 2. The architecture of the proposed network and Tiny YOLO V3.

The proposed network					Tiny YOLOV3			
Layer	Type	Filters	Size/Stride/Dilated rate	Output	Type	Filters	Size/Stride	Output
0	Convolutional	16	3×3/1	512×512	Convolutional	16	3×3/1	512×512
1	Convolutional	32	3×3/2	256×256	Max Pooling		2×2/2	256×256
2	Convolutional	16	1×1/1	256×256	Convolutional	32	3×3/1	256×256
3	Convolutional	32	3×3/1	256×256	Max Pooling		2×2/2	128×128
4	Convolutional	64	3×3/2	128×128	Convolutional	64	3×3/1	128×128
5	Convolutional	32	1×1/1	128×128	Max Pooling		2×2/2	64×64
6	Convolutional	64	3×3/1	128×128	Convolutional	128	3×3/1	64×64
7	Convolutional	32	1×1/1	128×128	Max Pooling		2×2/2	32×32
8	Convolutional	64	3×3/1	128×128	Convolutional	256	3×3/1	32×32
9	Convolutional	32	1×1/1	128×128	Max Pooling		2×2/2	16×16
10	Dilated Convolutional	64	3×3/1/2	128×128	Convolutional	512	3×3/1	16×16
11	Convolutional	32	1×1/1	128×128	Max Pooling		2×2/1	16×16
12	Concatenation		Layer 11+Layer 9	128×128	Convolutional	1024	3×3/1	16×16
13	Convolutional	32	1×1/1	128×128	Convolutional	256	1×1/1	16×16
14	Dilated Convolutional	64	3×3/1/4	128×128	Convolutional	512	3×3/1	16×16
15	Convolutional	32	1×1/1	128×128	Convolutional	18	1×1/1	16×16
16	Concatenation		Layer 15+Layer 13	128×128	Yolo			
17	Convolutional	32	1×1/1	128×128	Route		Layer 13	16×16
18	Concatenation		Layer 9+Layer 13	128×128	Convolutional	128	1×1/1	16×16
19	Convolutional	32	1×1/1	128×128	Upsample			32×32
20	Concatenation		Layer 19+Layer 17	128×128	Route		Layer 19+Layer 8	32×32
21	Convolutional	128	3×3/2	64×64	Convolutional	256	3×3/1	32×32
22	Convolutional	64	1×1/1	64×64	Convolutional	N	1×1/1	32×32
23	Convolutional	128	3×3/1	64×64	Yolo			
24	Convolutional	64	1×1/1	64×64				
25	Convolutional	128	3×3/1	64×64				
26	Route		Layer 8	64×64				
27	Convolutional	16	1×1/1	64×64				
28	Passthrough		/2	64×64				
29	Concatenation		Layer 25+Layer 28	64×64				
30	Convolutional	128	1×1/1	64×64				
31	Convolutional	256	3×3/1	64×64				
32	Convolutional	N	1×1/1	64×64				

targets is poor. Our proposed network uses dilated modules to expand the receptive field instead of downsampled layers and applies feature fusion and passthrough module to merge the finer-grained information with the meaningful semantic information. What is more, 1×1 convolution is used to reduce the dimension which makes our network more efficient and fast.

The goal of our network is to detect the small targets. Multiple downsampled layers are not good for small targets detection. However, the number of the downsampled layers directly affects the size of the receptive field. The size of the receptive field determines the amount of the contextual information of the small targets. So we used two dilated modules (Dilated module A and Dilated module B) to expand the receptive field. Feature map downsampled twice from the input is used as finer-grained information and combined with the feature map downsampled three times by

passthrough module. To improve the speed of our network, the 1×1 convolution is used to reduce the dimension of the network after each concatenation. The final layer provides the result of the prediction which contains the location of the bounding box and the class of the targets, and the size of the last layer is $N = N_{boxes} \times (N_{classes} + 4 + 1)$ for the 4 bounding box offsets and 1 objectness prediction, where N_{boxes} is the number of boxes per grid (3 by default) in the feature map. $N_{classes}$ is the number of object classes [23].

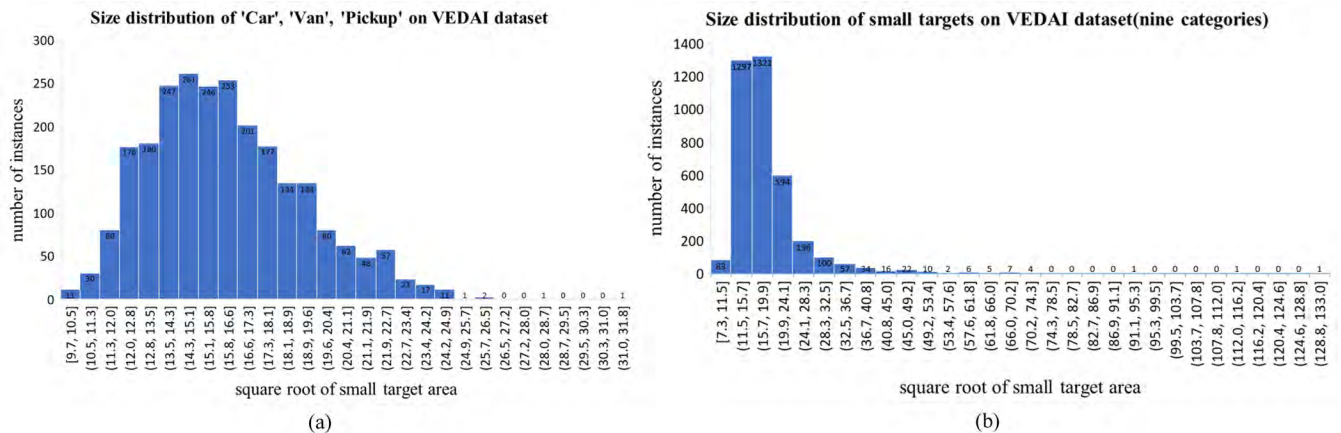
We calculate the receptive field by the following formula.

$$RF_k = RF_{k-1} + \left((f_k - 1) \times \prod_{i=1}^{k-1} s_i \right) \quad (1)$$

where RF_k is the receptive field of layer k , RF_{k-1} is the receptive field of layer $k - 1$, f_k is the filter size, and s_i is the stride of layer i . Compared the receptive field between the network

TABLE 3. Comparison of receptive field with different network.

Layer	Network	Size/Stride	RF	Layer	Network	Size/Stride/Dilated rate	RF
0	Convolutional	3×3/1	3×3	0	Convolutional	3×3/1	3×3
1	Convolutional	3×3/2	5×5	1	Convolutional	3×3/2	5×5
2	Convolutional	3×3/1	9×9	2	Convolutional	3×3/1	9×9
3	Convolutional	3×3/2	13×13	3	Convolutional	3×3/2	13×13
4	Convolutional	3×3/1	21×21	4	Convolutional	3×3/1	21×21
5	Convolutional	3×3/2	29×29	5	Dilated Convolutional	3×3/1/2	37×37
6	Convolutional	3×3/1	45×45	6	Dilated Convolutional	3×3/1/4	69×69
7	Convolutional	3×3/2	61×61	7	Convolutional	3×3/2	77×77
8	Convolutional	3×3/1	93×93	8	Convolutional	3×3/1	93×93

**FIGURE 8.** The distribution of the targets on VEDAI dataset. (a) Three categories. (b) Nine categories.

downsampled 4 times and the network downsampled 3 times with two dilated convolutions, the results are shown in Table 3. With two dilated convolutions, the network downsampled 3 times has the same receptive field (RF) as the network downsampled 4 times. It demonstrates that dilated convolution can expand the receptive field effectively.

To improve the speed and the efficiency of the proposed network, we used 1×1 convolution to reduce the dimension of the model after each concatenation. Because small deviations in small targets matter more than in big targets, the location information is important to the small target detection. To get more finer-grained information such as location, we used passthrough layer to merge the features from the earlier layer with the features from the deeper layer.

III. EXPERIMENT

To demonstrate and evaluate the performance of the proposed network, the comparative experiments of our approach with state-of-the-art object detection algorithms were done on the vehicle datasets we made based on VEDAI dataset and DOTA dataset respectively.

A. DATASET DESCRIPTION AND ANALYSIS

The dataset we used to evaluate the performance of our network is based on VEDAI dataset and DOTA dataset.

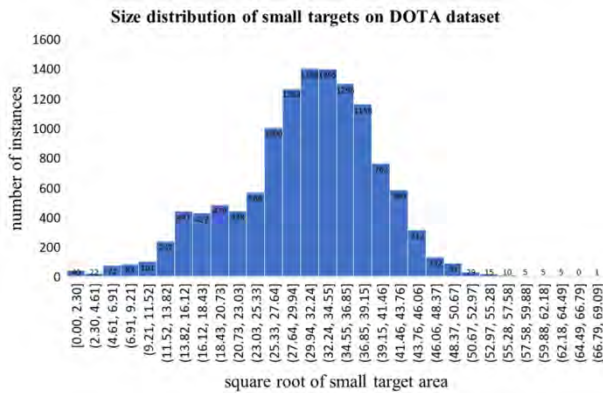
The targets in VEDAI dataset are smaller than the ones in DOTA dataset. However, the quantity of the targets per image in DOTA dataset is much more than that of VEDAI.

VEDAI is a dataset for vehicle detection in Aerial Imagery. The original large aerial images were segmented into 1024×1024 , containing various vehicles and backgrounds. All images have been taken from the same distance to the ground. The images are also downsampled to 512×512 to make the targets smaller. VEDAI dataset contains nine different classes of vehicles, namely the 'plane', 'boat', 'camping car', 'car', 'pick-up', 'tractor', 'truck', 'van', and the 'other' category. There is an average of 5.5 vehicles per image, and they occupy about 0.7% of the total pixels of the image [31].

In this paper, we composed two vehicles dataset based on VEDAI dataset. To make the detection of the small targets more challenging, we chose the images whose size is 512×512 in VEDAI dataset. Dataset 1: we merged 'car', 'pick-up', and 'van' which are the smallest vehicles in VEDAI dataset as one category 'car'. The distribution of this dataset is shown in Fig. 8(a). The size of the targets is concentrated from 12×12 to 19.6×19.6 . Dataset 2: we evaluated the performance of our model on the nine categories of VEDAI dataset. The distribution of this dataset is shown in Fig. 8(b). The size of the targets is concentrated from 11.5×11.5 to 24.1×24.1 .

TABLE 4. The results of each detection algorithm in experiment 1.

Detection algorithm	Input	TP	FP	FN	P	R	AP
YOLO V2	512×512	283	296	147	48.9%	65.8%	57.33%
Tiny YOLO V3	512×512	307	305	123	50.2%	71.4%	58.17%
YOLO V3	512×512	373	69	57	84.3%	86.7%	85.37%
Ours	512×512	362	88	68	80.5%	84.2%	80.16%

**FIGURE 9.** The distribution of small-vehicle on DOTA dataset.

DOTA is a large-scale dataset for object detection in aerial images. It contains 2806 aerial images from different sensors and platforms. Each image is of the size in the range from about 800×800 to 4000×4000 pixels and contains objects exhibiting a wide variety of scales, orientations, and shapes. It contains 15 common object categories [32].

As the size of the images in DOTA dataset are not uniform. We cropped the images to 512×512 . The targets in DOTA dataset are almost too big, such as ‘Bridge’, ‘Soccer-ball field’, ‘Basketball court’, ‘Harbor’ and so on. Therefore, we chose the single category ‘Small-vehicle’ as the small targets. Finally, we selected manually 848 images that contain ‘small-vehicle’ category as the final dataset. There is an average of 15 vehicles per image which is three times as many as the ones in VEDAI dataset. The distribution of this dataset is shown in Fig.9. The size of the targets is concentrated from 25.3×25.3 to 41.5×41.5 which is bigger than the ones in VEDAI dataset.

B. EVALUATION METRIC

The prediction is considered correctly if the intersection over union (IOU) is greater than 0.5. We used recall rate, precision rate and average precision (AP) to evaluate the performance of our approach when there is only one category.

$$R = \frac{TP}{TP + FN} \quad (2)$$

$$P = \frac{TP}{TP + FP} \quad (3)$$

where TP means that positive samples were detected as positive samples. FP means that negative samples were detected as positive samples. FN means that positive samples were detected as negative samples. R represents recall rate of the targets. P represents precision rate of the targets.

Average precision evaluates the accuracy of detection from the perspective of recall rate and precision rate. It can be used to analyze the detection performance of a single category.

When the category of the targets is more than one, we can compute the mean average precision (mAP) by the precision and recall curve to evaluate the performance of our model.

We used frames per second (FPS) to test the detection speed and used billions of floating point operations (BFLOPs) to calculate the computation for our network.

C. IMPLEMENTATION DETAILS OF THE EXPERIMENT

For fair comparison, our proposed method was implemented in Darknet which is the same as YOLO V3 and tiny YOLO V3. All the models are trained with 512×512 input on the same dataset. We performed different methods using an Intel i7-5930k processor with 64GB RAM and TITAN X GPU.

On VEDAI dataset, the batch size, the subdivisions and the initial learning rate were set to 64, 8 and 0.001. Then the learning rate was reduced to 0.0001 and 0.00001 for 80000 and 100000 batches. On DOTA dataset, the batch size, the subdivisions and the initial learning rate were set to 64, 8 and 0.001. The learning rate was reduced to 0.0001 and 0.00001 for 100000 and 120000 batches.

D. EXPERIMENT ON VEDAI DATASET

Experiment 1: Test the model on the dataset which merged ‘car’, ‘pick-up’, and ‘van’ as one category ‘car’. The experimental results of each algorithm are shown in Table 4. The recall rate, precision rate and AP of ‘Ours’ for 512×512 input models was 80.5%, 84.2% and 80.16%. Compared with YOLO V2, the recall rate, precision rate and AP has been improved by 18.4%, 31.6% and 22.83% respectively. The performance of our model has been improved obviously. Compared with Tiny YOLO V3, the recall rate, precision rate and AP has been improved by 15.8%, 23.5% and 21.27% respectively. The recall rate, precision rate and AP of YOLO V3 for 512×512 input models was 84.3%, 86.7% and 85.37%

TABLE 5. The results of each detection algorithm in experiment 2.

Detection algorithm	AP/%									mAP/%
	Boat	Camping	Car	Others	Pickup	Tractors	Truck	Vans	Plane	
YOLO V2	4.4	44	42	11.8	24.5	11.8	18.7	1.05	0	17.6
Tiny YOLO V3	3.3	25.1	41.2	8.9	24	16.7	22.5	19.4	0	17.8
YOLO V3	21	61	78	22	70	43	55	52	100	55.8
Ours	26.5	54.8	60	19	52	37	24.5	56.6	100	47.8

TABLE 6. The results of each algorithm on DOTA dataset.

Detection algorithm	Input	TP	FP	FN	P	R	AP
YOLO V2	512×512	1472	438	439	77%	77%	72.74%
Tiny YOLO V3	512×512	1557	536	354	74.4%	81.5%	73.2%
YOLO V3	512×512	1750	228	161	88.47%	91.6%	88.31%
Ours	512×512	1753	278	158	86.5%	91.7%	88.63%

TABLE 7. Comparative results of FPS, BFLOPs and Model Size with different algorithms.

Object detection algorithm	YOLO V2	Tiny YOLO V3	YOLO V3	Ours
Input	512×512	512×512	512×512	512×512
Model Size	202.3 M	34.7 M	246.3 M	2.8 M
FPS	58.3	76.4	14.7	75.4
BFLOPs	44.417	8.243	101.784	9.692

The detection performance of our network on small targets can almost catch up with that of YOLO V3.

Experiment 2: Test the model on the dataset which contains the nine categories of VEDAI dataset. The experimental results of each algorithm are shown in Table 5. The mAP of ‘Ours’ for 512×512 input models was 47.8%. Compared with YOLO V2, mAP has been improved by 22.83%. Compared with Tiny YOLO V3, the mAP has been improved by 21.27%. The detection performance of our network on small targets can almost catch up with that of YOLO V3.

E. EXPERIMENT ON DOTA DATASET

Test the model on the dataset we made based on DOTA dataset. The experimental results of each algorithm are shown in Table 6. The recall rate, precision rate and AP of ‘Ours’ for 512 × 512 input models was 91.7%, 86.5% and 88.63%. Compared with YOLO V2, the recall rate, precision rate and AP has been improved by 14%, 11.7% and 15.38% respectively. Compared with Tiny YOLO V3, the recall rate, precision rate and AP has been improved by 23.3%, 29.5% and 29.74% respectively. The performance of our model has been improved obviously. The recall rate, precision rate and

AP of YOLO V3 for 512 × 512 input models was 91.6%, 88.47% and 88.31%. The mAP of our network is even better than that of YOLO V3 on DOTA dataset. The experimental results demonstrated that the detection performance of our network on small targets can almost catch up with that of YOLO V3.

F. FPS AND BFLOPs FOR DIFFERENT NETWORK

Comparative experiments have been done to evaluate the detection speed and the computation of each network. The experimental results of each network are shown in Table 7. The FPS of our network is 75.4 which is 5 times as many as that of YOLO V3 and almost catches up with the speed of tiny YOLO V3. The BFLOPs of our model is 9.692 which is nearly the same as the result of tiny YOLO V3. The BFLOPs of YOLO V3 is almost ten times as many as that of our network. The model size of our network is much smaller than tiny YOLO V3. Because we used 1 × 1 convolution to reduce the dimension of the network, there are much less parameters in our model. As shown in Table 2, the numbers of filters we used in 3 × 3 convolution are 256, 128, 64 and 32. However, the numbers of filters used in Tiny YOLO V3 are 1024, 512, 256, 128, and 64 which are much more than our proposed



FIGURE 10. Comparison of different algorithms on VEDAI dataset. (a) YOLO V2. (b) Tiny YOLO V3. (c) YOLO V3. (d) Ours.

network. The experimental results have demonstrated that the proposed network is really simple and efficient.

G. QUANTITATIVE AND QUALITATIVE EVALUATION

1) QUANTITATIVE EVALUATION

With the experimental results on VEDAI dataset and DOTA dataset, it shows that the detection performance of our network is much better than that of tiny YOLO V3 and YOLO V2. The detection ability on small targets of our network can almost catch up with that of YOLO V3. However, the computation of our network is much less than the result of YOLO V3. The FPS of the proposed network is almost the same as the result of tiny YOLO V3. All of the results above demonstrated that our network achieves excellent performance on small targets.

2) QUALITATIVE EVALUATION

We also provide a qualitative comparison of our network with YOLO V2, tiny YOLO V3 and YOLO V3. Fig.10 and Fig.11 shows the detection results on VEDAI dataset and

DOTA dataset respectively and the backgrounds in different images are also different. The light blue box is the ground truth. The green box is the prediction box. The red box is the targets of the fault detection. It is obvious that YOLO V3 and our network achieved a better performance than YOLO V2 and tiny YOLO V3 on small targets.

In Fig.10 and Fig.11, a lot of targets are missed and detected with fault by YOLO V2 and tiny YOLO V3. These problems can be avoided by the proposed network and YOLO V3. That is because ‘diluted module’ in our proposed network can increase the contextual information surrounding the small targets by expanding the receptive field. The ability to detect the small targets of our network is improved. What is more, with feature fusion and ‘passthrough module’, the proposed network can locate the small targets more accurately. Compared Fig 11(c) with Fig 11(d), there are less fault targets detected by our method than the ones detected by YOLO V3. It demonstrates that the detection performance of our method is even better than YOLO V3 which has the same conclusion as the quantitative evaluation on DOTA dataset.

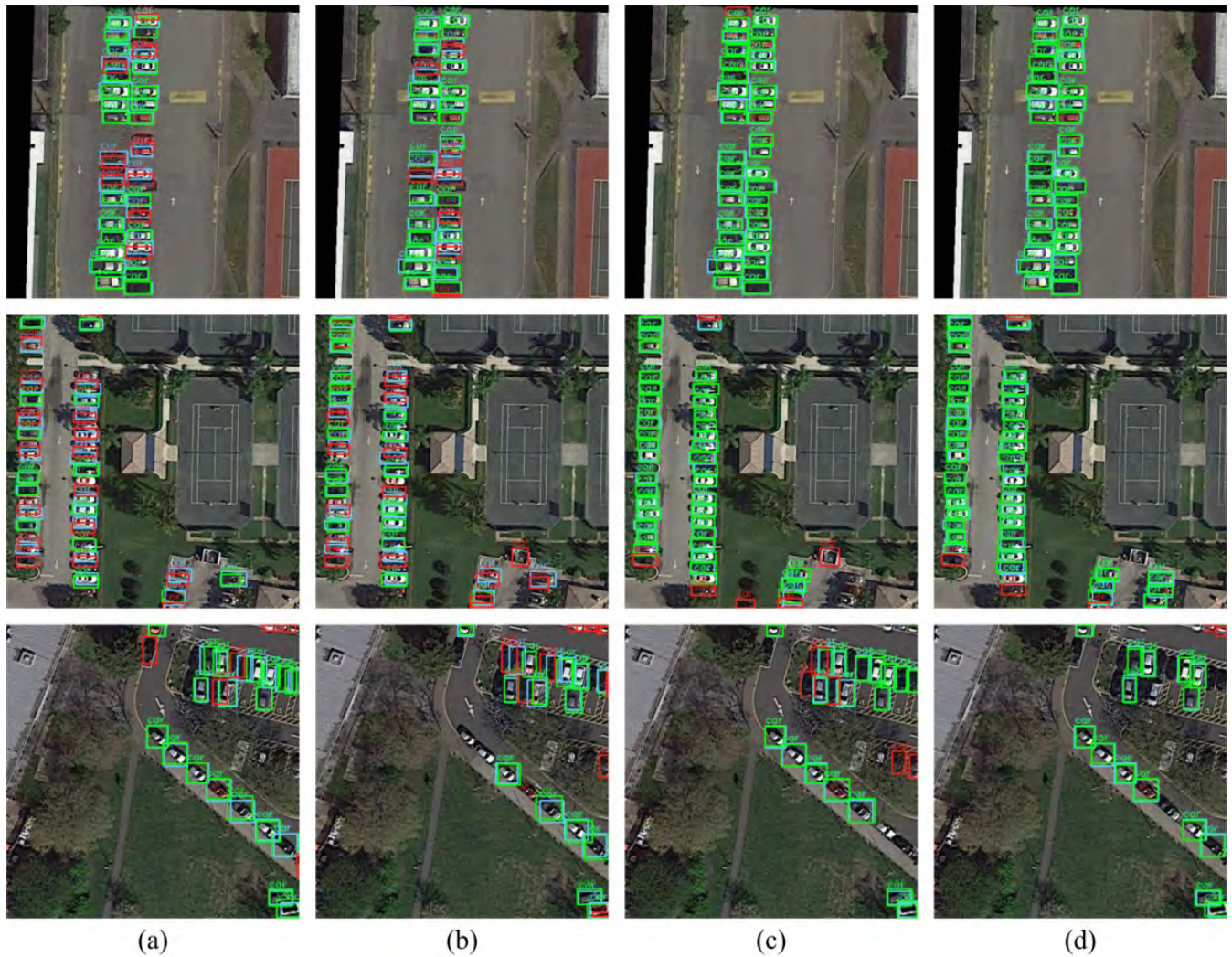


FIGURE 11. Comparison of different algorithms on DOTA dataset. (a) YOLO V2. (b) Tiny YOLO V3. (c) YOLO V3. (d) Ours.

IV. CONCLUSION

In this paper, a simple, fast and efficient network is proposed and applied to small target detection. The main contributions of the proposed network are as follows: First, too many downsampled layers is not favorable for detecting the small targets. We proposed ‘dilated module’ to expand the receptive field without loss of resolution or coverage. Second, we used feature fusion in different dilated module to make good use of the features of the small targets. What is more, ‘passthrough module’ is proposed to merge the finer-grained information from the earlier layer with the meaningful semantic information from the deeper layer. Finally, we made small vehicle dataset based on VEDAI dataset and DOTA dataset and analyzed the distribution of the small targets in each dataset. Our experiments on the dataset above provided both quantitative and qualitative comparison of our network with the state-of-the-art networks. The results show that the detection performance of our network is much better than that of tiny YOLO V3 and YOLO V2. The detection ability on

small targets of our network can almost catch up with that of YOLO V3. However, the computation of our network is much less than the result of YOLO V3. The FPS of the proposed network is almost the same as the result of tiny YOLO V3. It demonstrated that the proposed network is really simple, fast and efficient.

REFERENCES

- [1] L. Liu, W. Ouyang, X. Wang, P. Fieguth, J. Chen, X. Liu, and M. Pietikäinen, “Deep learning for generic object detection: A survey,” 2018, *arXiv:1809.02165*. [Online]. Available: <https://arxiv.org/abs/1809.02165>
- [2] J. Han, D. Zhang, G. Cheng, N. Liu, and D. Xu, “Advanced deep-learning techniques for salient and category-specific object detection: A survey,” *IEEE Signal Process. Mag.*, vol. 35, no. 1, pp. 84–100, Jan. 2018.
- [3] Y. Tian, P. Luo, X. Wang, and X. Tang, “Pedestrian detection aided by deep learning semantic tasks,” 2015, *arXiv:1412.0069*. [Online]. Available: <https://arxiv.org/abs/1412.0069>
- [4] G. Levi and T. Hassner, “Age and gender classification using convolutional neural networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops. (CVPRW)*, Boston, MA, USA, Jun. 2015, pp. 34–42.

- [5] A. Attia and S. Dayan, "Detecting and counting tiny faces," 2018, *arXiv:1801.06504*. [Online]. Available: <https://arxiv.org/abs/1801.06504>
- [6] Y. Zhou, L. Liu, L. Shao, and M. Mellor, "DAVE: A unified framework for fast vehicle detection and annotation," 2016, *arXiv:1607.04564*. [Online]. Available: <https://arxiv.org/abs/1607.04564>
- [7] L. Wang, Y. Lu, H. Wang, Y. Zheng, H. Ye, and X. Xue, "Evolving boxes for fast vehicle detection," 2017, *arXiv:1702.00254*. [Online]. Available: <https://arxiv.org/abs/1702.00254>
- [8] K. Liu and G. Mattyus, "Fast multiclass vehicle detection on aerial images," *IEEE Geosci. Remote Sens. Lett.*, vol. 12, no. 9, pp. 1938–1942, Sep. 2015.
- [9] L. T. Nguyen-Meidine, E. Granger, M. Kiran, and L.-A. Blais-Morin, "A comparison of CNN-based face and head detectors for real-time video surveillance applications," 2018, *arXiv:1809.03336*. [Online]. Available: <https://arxiv.org/abs/1809.03336>
- [10] R. Yu, H. Wang, and L. S. Davis, "ReMotENet: Efficient relevant motion event detection for large-scale home surveillance videos," 2018, *arXiv:1801.02031*. [Online]. Available: <https://arxiv.org/abs/1801.02031>
- [11] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1798–1828, Aug. 2013.
- [12] X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, "Multi-view 3D object detection network for autonomous driving," 2016, *arXiv:1611.07759*. [Online]. Available: <https://arxiv.org/abs/1611.07759>
- [13] A. Uçar, Y. Demir, and C. Güzelic, "Object recognition and detection with deep learning for autonomous driving applications," *Simulation*, vol. 93, no. 9, pp. 759–769, 2017.
- [14] G. Cao, X. Xie, W. Yang, Q. Liao, G. Shi, and J. Wu, "Feature-fused SSD: Fast detection for small objects," 2018, *arXiv:1709.05054*. [Online]. Available: <https://arxiv.org/abs/1709.05054>
- [15] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Columbus, OH, USA, Jun. 2014, pp. 580–587.
- [16] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1440–1448.
- [17] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.
- [18] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Oct. 2017, pp. 2980–2988.
- [19] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. Lecun, "OverFeat: Integrated recognition, localization and detection using convolutional networks," Dec. 2013, *arXiv:1312.6229*. [Online]. Available: <https://arxiv.org/abs/1312.6229>
- [20] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Computer Vision—ECCV*. Cham, Switzerland: Springer, 2016, pp. 21–37.
- [21] C.-Y. Fu, W. Liu, A. Ranga, A. Tyagi, and A. C. Berg, "DSSD: Deconvolutional single shot detector," 2017, *arXiv:1701.06659*. [Online]. Available: <https://arxiv.org/abs/1701.06659>
- [22] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.
- [23] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 6517–6525.
- [24] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*. [Online]. Available: <https://arxiv.org/abs/1804.02767>
- [25] A. Van Etten, "You only look twice: Rapid multi-scale object detection in satellite imagery," 2018, *arXiv:1805.09512*. [Online]. Available: <https://arxiv.org/abs/1805.09512>
- [26] C. Chen, M.-Y. Liu, O. Tuzel, and J. Xiao, "R-CNN for small object detection," in *Proc. Asian Conf. Comput. Vis.*, 2017, pp. 214–230.
- [27] C. Eggert, D. Zecha, S. Brehm, and R. Lienhart, "Improving small object proposals for company logo detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 167–174.
- [28] P. Hu and D. Ramanan, "Finding tiny faces," 2017, *arXiv:1612.04402*. [Online]. Available: <https://arxiv.org/abs/1612.04402>
- [29] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," 2015, *arXiv:1511.07122*. [Online]. Available: <https://arxiv.org/abs/1511.07122>
- [30] J. Leng and Y. Liu, "An enhanced SSD with feature fusion and visual reasoning for object detection," in *Neural Computing and Applications*. London, U.K.: Springer, 2018.
- [31] S. Razakarivony and F. Jurie, "Vehicle detection in aerial imagery: A small target detection benchmark," *J. Vis. Commun. Image Represent.*, vol. 34, pp. 187–203, Jan. 2016.
- [32] G.-S. Xia, X. Bai, J. Ding, Z. Zhu, S. Belongie, J. Luo, M. Datcu, M. Pelillo, and L. Zhang, "DOTA: A large-scale dataset for object detection in aerial images," 2018, *arXiv:1711.10398*. [Online]. Available: <https://arxiv.org/abs/1711.10398>



MORAN JU was born in Jilin, Jilin, China, in 1989. He received the M.S. degree from Jilin University. He is currently pursuing the Ph.D. degree with the University of Chinese Academy of Sciences. His current research includes deep learning, target detection, and small target detection.



JIANGNING LUO received the B.S. degree from the University of Macau, in 2018. He is currently pursuing the M.Eng. degree with McGill University. He currently does research on control systems along with H. Michalska.



PANPAN ZHANG was born in Yucheng, Henan, China, in 1992. He is currently pursuing the Ph.D. degree with the University of Chinese Academy of Sciences. His current research interests include deep learning and brain-inspired learning.



MIAO HE received the B.S. degree from the College of Electronic Information and Optical Engineering, Nankai University, China, in 2015. He is currently pursuing the Ph.D. degree with the Shenyang Institute of Automation, Chinese Academy of Sciences. His currently research interests include target detection, semantic segmentation, multi-target tracking, and pedestrian re-identifying.



HAIBO LUO received the M.S. degree from the Harbin Institute of Technology, in 1990, and the Ph.D. degree from the Shenyang Institute of Automation, Chinese Academy of Sciences, in 2009. He is currently a Professor with the Shenyang Institute of Automation. His research interests include target detection, target tracking, and image processing.

...